# A Practical Guide to Retrieval-Augmented Generation (RAG)

Understanding how document-based AI assistants find information and generate grounded answers

## 1. What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with a language model. Instead of relying only on information learned during training, a RAG application searches a supplied knowledge collection and gives the most relevant passages to the model as context.

For example, a company could place its policy manuals in a document collection. When an employee asks, “How many days of annual leave are available?”, the system retrieves the relevant policy section and asks the language model to answer using that section.

## 2. Why use RAG?

A language model may not know the contents of a private document, and its general knowledge may be outdated or incomplete. RAG helps connect the model to a specific, changeable source of information.

- **Document-specific answers:** responses can be based on manuals, reports, research papers, or other supplied files.
- **More current information:** update the document collection without retraining the language model.
- **Traceability:** retrieved passages can be associated with filenames and page numbers.
- **Focused generation:** instructions can tell the model to use retrieved context and acknowledge when an answer is missing.

## 3. The main stages of a RAG pipeline

A typical RAG workflow has two broad phases: preparing the knowledge collection and answering a user's question.

### A. Document preparation

- **Load:** read supported files such as PDF or TXT and extract their text and metadata.
- **Split:** divide long text into smaller, overlapping chunks so relevant passages can be retrieved efficiently.
- **Embed:** convert each chunk into a numerical vector that represents its semantic meaning.
- **Store:** save vectors, text, and metadata in a vector database such as ChromaDB or FAISS.

### B. Question answering

- **Receive a question:** the user enters a natural-language query.
- **Retrieve:** embed the query and find chunks with similar vector representations.
- **Build context:** combine the selected chunks and their source metadata.
- **Generate:** send the question and retrieved context to a language model, which produces an answer.

*The quality of the final answer depends on every stage. Poor text extraction, unsuitable chunk sizes, weak retrieval, or unclear instructions can all reduce accuracy.*

# Making RAG Reliable in Practice

Key design choices, an example, and common limitations

## 4. Important components

Document loaders extract text and metadata. For PDFs, metadata should retain the source filename and original page index. A page index that starts at zero should be converted to a human-readable page number by adding one.

Text splitters control chunk size and overlap. Small chunks can lose context, while very large chunks may contain unrelated information. Overlap helps preserve meaning when a useful passage crosses a chunk boundary.

Embedding models map text into vectors. A vector store indexes those vectors and supports similarity search. The retrieved results should include the original text and metadata, not just the vector values.

The language model receives the question and retrieved passages. A clear instruction should require it to answer from those passages only and say when the documents do not contain the answer.

## 5. Example: asking a question about a PDF

**Document:** A three-page guide explains supervised, unsupervised, and reinforcement learning.

**Question:** “What are the three types of machine learning described in the guide?”

**Retrieved context:** The search returns the paragraph describing supervised learning from page 1 and the paragraphs describing unsupervised and reinforcement learning from page 2.

**Grounded answer:** “The guide describes supervised learning, unsupervised learning, and reinforcement learning.”

**Sources:** learning-guide.pdf (Page 1), learning-guide.pdf (Page 2)

> *This example shows why retrieval must return all passages needed to answer a question. If one of the three types is missing from the retrieved context, the model may produce an incomplete answer even when the information exists elsewhere in the PDF.*

## 6. Common challenges and practical improvements

- **Scanned PDFs:** image-only pages may contain no extractable text. Optical character recognition (OCR) is needed before chunking.
- **Incorrect retrieval:** increase or tune the number of retrieved chunks, improve chunk boundaries, or use keyword and vector search together.
- **Duplicate content:** use stable document and chunk identifiers so re-indexing does not repeatedly add identical records.
- **Unsupported questions:** set a relevance threshold or validate retrieved context before asking the model to answer.
- **Weak citations:** preserve source and page metadata through loading, splitting, storage, and retrieval.

## 7. How to evaluate a RAG assistant

Test the system with questions whose answers are known from the source documents. Include questions that require information from more than one page, questions about different files, and questions whose answers are not present.

- Check whether the correct chunks are retrieved, not only whether the final answer sounds plausible.
- Verify that every citation points to the correct file and page.
- Confirm that unsupported questions receive a clear “not found” response.

- Repeat tests after changing chunk settings, embeddings, or retrieval parameters.

## Key takeaway

RAG does not guarantee that a language model will always be correct. It provides a structured way to ground answers in supplied documents. Reliable results require good extraction, thoughtful chunking, relevant retrieval, clear generation instructions, and verifiable citations.